

Implementation Plan: Clinical Rehab Games, Patient Profiles & Fir

This plan outlines the architecture, data structures, state machines, and task distribution required to implement the new clinical rehabilitation games and patient-centric database storage.

The development work is divided evenly among three developers:

1. **Ami**: ESP32 Firmware & Low-Level Hardware Integration (audio driver, state machines, LEDs, haptics, box calibration flashing).
 2. **Ido**: Flutter Frontend UI/UX, State Management, Box/Sensor Calibration, and Therapist Configuration interfaces.
 3. **Dara**: Firebase/Firestore Database Architecture, Glove Offline Buffer, Sync Service, and Local Network Failover.
-

Technical Architecture & Design Decisions

1. Patient-Centric Firestore Schema

We will transition to **Cloud Firestore** for structured patient profiles, therapy configurations (prescriptions), and game history logs.

Firestore Collections:

- **`patients`** (Collection)
 - **`id`** (String): Unique patient ID.
 - **`name`** (String): Patient's full name.
 - **`createdAt`** (Timestamp): Registration date.
 - **`calibration`** (Map): Current sensor calibration coefficients (FSR curve coefficients, Flex limits).
 - **`prescription`** (Map): Active exercise parameters:
 - * **`cubesBoxes`**: `{ timer: 60, cycles: 3, allowedWeightsGrams: [100, 500], difficulty: 2 }`
 - * **`pinch`**: `{ targetWeightGrams: 100, requiredHoldTimeSeconds: 5, activeFingers: [1, 2] }`
 - * **`bend`**: `{ allowedTimeSeconds: 30, sequence: [1, 2, 3, 4, 5], requiredRom: [80, 80, 80, 80, 80] }`
- **`game\_history`** (Collection)
 - **`id`** (String): Document ID (auto-generated).
 - **`patientId`** (String): Reference to the patient.
 - **`gameType`** (String): `"CubesBoxes" | "Pinch" | "Bend"`.
 - **`timestamp`** (Timestamp): Completion date and time.
 - **`successCount`** (Integer): Completed goals.
 - **`totalCycles`** (Integer): Programmed cycles (representing the number of times the sequence of transferring all designated cubes repeats within a session).
 - **`metrics`** (Map): Game-specific stats:
 - * **`CubesBoxes`**: `{ avgResponseTimeSeconds: 2.4, levelCompleted: 2 }`
 - * **`Pinch`**: `{ avgSteadyStateForceGrams: 320, maxHoldTimeSeconds: 4.8, succeeded: true }`
 - * **`Bend`**: `{ avgRomReached: [85, 90, 75, 80, 82], sequenceCompleted: true }`
- **`cubes`** (Collection)
 - **`uid`** (String, Doc ID): NFC Tag hex string (e.g., `"04AA88CC"`).
 - **`name`** (String): Name assigned (e.g., `"Circle Cube Light"`).
 - **`color`** (String): Display color (e.g., `"Red"`, `"Blue"`).
 - **`shape`** (String): Physical shape (e.g., `"Circle"`, `"Triangle"`, `"Square"`).
 - **`weightGrams`** (Integer): Calibrated physical weight.
- **`boxes`** (Collection)

- ``macAddress`` (String, Doc ID): MAC address hex representation.
 - ``name`` (String): Named identifier (e.g., ``"Target Box A"``).
 - ``shape`` (String): Physical slot shape (e.g., ``"Circle"``, ``"Triangle"``, ``"Square"``).
-

2. Force & Sensor Calibration Flow (Online vs. Offline)

- **Grams Conversion Algorithm:**

1. The therapist puts sample weights (e.g., 0g, 100g, 500g) on the FSR force sensor.
2. The Flutter app reads the raw ADC values from the Glove.
3. The Flutter app fits an interpolation curve (linear or polynomial) to calculate the calibration coefficients.

- **Online Calibration:** The Flutter app uploads these coefficients to the patient's Firestore profile, which then syncs down to the Glove.

- **Offline Calibration (Direct Local Link):**

1. If internet is down, the Flutter app connects directly to the Glove via local Wi-Fi.
 2. The app fetches raw sensor values from ``GET /api/raw-sensors``.
 3. The app computes the interpolation coefficients locally.
 4. The app sends the calculated coefficients directly to the Glove via ``POST /api/calibrate-sensor``.
 5. The Glove saves these coefficients locally in its NVS flash memory (``Preferences``), allowing it to calculate grams locally for the active prescription.
 6. Once internet is restored, the Flutter app syncs these new calibration parameters up to the patient's Firestore profile.
-

3. Offline Log Persistence & Synchronization

- **Glove as the Primary Buffer:** When a game session is completed, the Glove generates the session results JSON.
 - **Offline Logging:** If the Glove has no internet connection, it writes the session logs directly to its local NVS/SPIFFS flash memory. The Flutter app does **not** poll or buffer these logs.
 - **Online Reconnection Sync:** The Glove monitors its internet connectivity. As soon as a connection is established, the Glove uploads the buffered logs directly to the Firestore database (via Cloud Functions or Firestore REST API), ensuring no data is lost.
-

4. Direct Local Wi-Fi Routes (Fallback)

When offline, the Glove web server serves as the command receiver:

- ``POST /api/active-prescription``: Sends the active therapist prescription directly to the Glove.
 - ``GET /api/raw-sensors``: Exposes raw ADC values of force and flex sensors for real-time calibration.
 - ``POST /api/calibrate-sensor``: Submits calculated calibration coefficients directly to the Glove's local NVS storage.
-

Detailed Task Division (Even 3-Way Split)

Ami: Firmware & Hardware Integration

Ami is responsible for the low-level ESP32 Glove and Smart Box firmware:

- **Task A.1: Audio Driver Integration:** Connect and initialize the DFPlayer Mini MP3 hardware module via UART.

Write helper functions like ``playAudioCue(int trackId)`` and ``playBeep()``.

- **Task A.2: Game State Machines (Glove):** Implement the local C++ state machines for:

1. *Cubes & Boxes*: Level 1, 2, and 3 logic (NFC validation, cycle counter, target matching, response timer).
2. *Pinch*: Monitor FSR pressure during cube-lift state (NFC absent from box), verify target threshold hold-time.
3. *Bend*: Monitor flex sensors to track sequence compliance and target ROM limit.

• **Task A.3: ESP-NOW Commands & LEDs:** Extend the ESP-NOW communication protocol to send LED states to the Smart Boxes (success flash, failure blink, target color) and trigger the Glove's haptic motor.

• **Task A.4: Box Identification Flash:** Implement support for parsing a specific box identification command. When triggered, the targeted Box will flash its LEDs to assist registration.

Dara: Connectivity, Offline Storage & Sync

Dara is responsible for the cloud databases, synchronization, and failover logic:

- **Task D.1: Firestore Collections & Rules:** Create the schema structure (``patients``, ``game_history``, ``cubes``, ``boxes``) in Firestore and configure security rules.
- **Task D.2: Firestore Flutter Integration Service:** Write the Flutter services to read/write configurations, profiles, and history logs directly to Firestore. Enable and configure local Firestore persistence.
- **Task D.3: ESP32 Offline Buffer System:** Implement flash memory buffering (Preferences/SPIFFS) on the Glove to store completed game payloads when offline.
- **Task D.4: ESP32 Firestore Sync Bridge:** Implement the C++ client that reads buffered logs and uploads them via secure REST API (or Cloud Functions) when Wi-Fi is back online.
- **Task D.5: Local Wi-Fi Fallback Endpoints:** Implement the local Glove Web Server routes (``POST /api/active-prescription``, ``GET /api/raw-sensors``, ``POST /api/calibrate-sensor``) to receive configurations and calibration parameters when offline.

Ido: Flutter UI/UX & Therapist Dashboard

Ido is responsible for the Flutter application screens and user experience:

- **Task I.1: Patient & Prescription Management UI:** Build the screen to list/add patients, select the active patient, and edit their configurable exercise prescriptions (timers, cycles, target weight, target ROM).
- **Task I.2: Active Exercise Control Screen:** Create the gameplay screen showing active instructions, countdown timers, cycle counts, and a trigger to start the exercise.
- **Task I.3: Analytics & Reporting Charts:** Develop a dashboard page displaying progress charts over time (success rate lines, response time curves, grip force changes, finger ROM flex charts).
- **Task I.4: Calibration Screen:** Build the interface to calibrate the Glove FSR against a digital scale (performing interpolation mapping of raw ADC values to grams) and map UUIDs to named objects. Supports direct local polling when offline.
- **Task I.5: Box Calibration UI:** Build the interface to trigger the Box identification LED flash and map Box MAC addresses to shape and name identities in the registry.